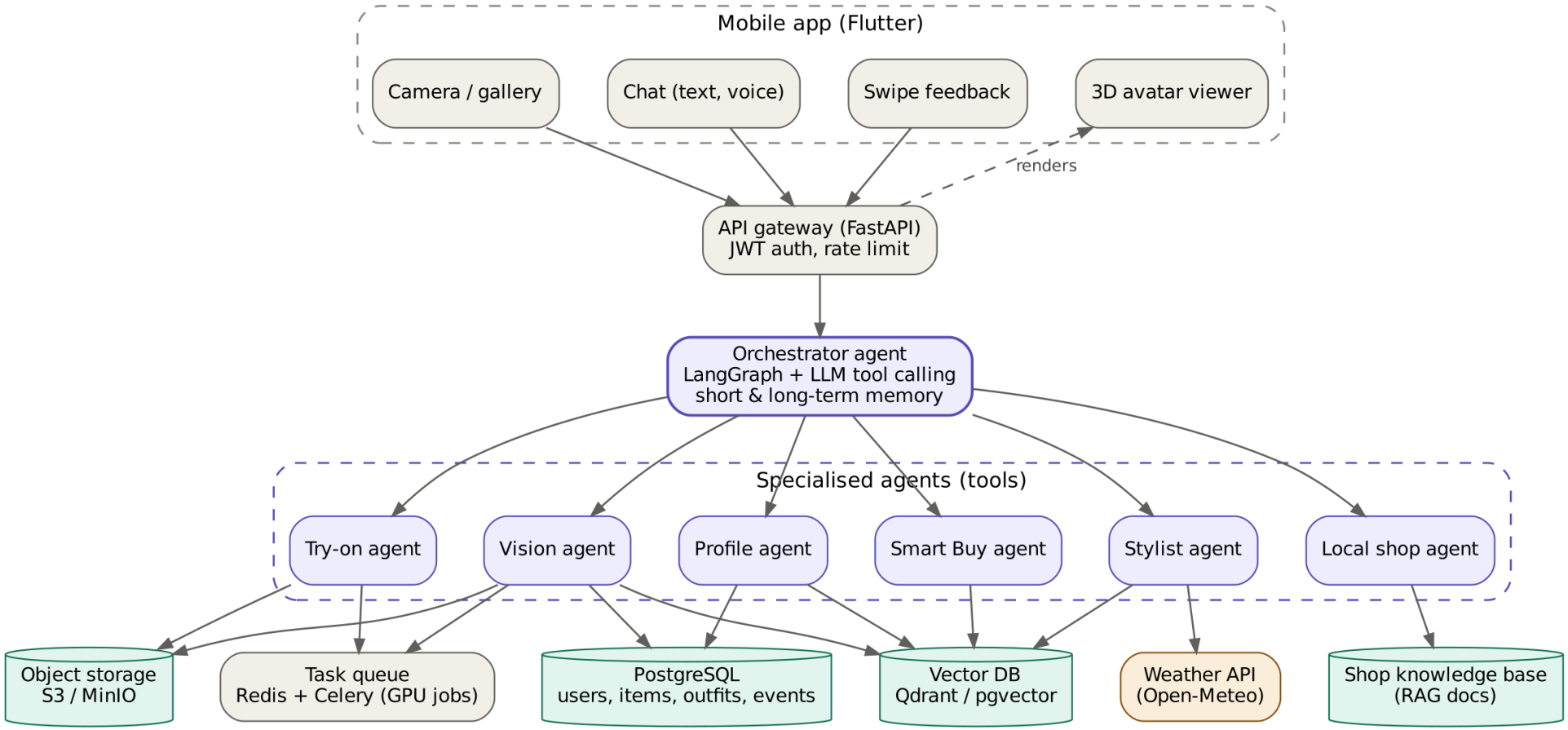
DressMe — Multi-agent pipeline, data & tech stack

Mobile app · agentic architecture · working document for the team of 6 (v1, 10 Oct 2026). Every request from the app reaches one **orchestrator agent**, which calls six specialised agents as tools. Agents share one data contract (garment, outfit, event) and four stores. Heavy GPU work (segmentation, try-on) runs asynchronously through a task queue.



1. Shared data contract (fix in week 1)

All agents read/write these three objects. Keep them versioned in one `schemas/` folder (Pydantic models on the backend, generated Dart classes in Flutter).

Garment

```
{
  "item_id": "itm_8f21",
  "user_id": "usr_042",
  "image_raw": "s3://dressme/raw/itm_8f21.jpg",
  "image_cutout": "s3://dressme/cut/itm_8f21.png",
  "category": "top",
  "subcategory": "shirt",
  "colors": [{"name":"navy","hex":"#1F2A44","share":0.82}],
  "pattern": "solid",
  "material": "linen",
  "season": ["spring","summer"],
  "formality": 0.7,
  "local_tag": null,
  "embedding_id": "vec_8f21",
  "confidence": 0.91,
  "wear_count": 4,
  "last_worn": "2026-10-02",
  "created_at": "2026-09-14T09:12:00Z"
}
```

Outfit

```
{
  "outfit_id": "out_77c",
  "user_id": "usr_042",
  "items": ["itm_8f21","itm_1a90",
            "itm_44de","itm_0b13"],
  "context": {
    "occasion": "interview",
    "date": "2026-10-11",
    "weather": {"temp_c":24,
                "condition":"sunny"}
  },
  "scores": {"compatibility":0.86,
             "style_match":0.79,
             "weather_fit":0.95},
  "explanation": "Navy linen shirt and
   beige chinos read formal but stay
   cool at 24°C.",
  "source": "stylist_agent",
  "tryon_render": "s3://dressme/vto/out_77c.glb"
}
```

Event

```
{
  "event_id": "evt_5521",
  "user_id": "usr_042",
  "type": "outfit_feedback",
  "target_id": "out_77c",
  "value": "accept",
  "channel": "swipe",
  "ts": "2026-10-10T07:41:00Z"
}

type ∈ {outfit_feedback,
        worn, item_added,
        item_deleted,
        tryon_reaction,
        builder_edit,
        smartbuy_check,
        chat_message}
```

2. Orchestrator agent (member 6)

Role. Understands the request, keeps conversation memory, decides which agents to call and in what order, merges results into one answer.

Example input

```
{ "user_id": "usr_042",
  "message": "What do I wear for an
   interview tomorrow?",
  "locale": "en", "city": "Tunis",
  "attachments": [] }
```

Example output

```
{ "intent": "outfit_request",
  "plan": ["get_weather",
           "get_profile",
           "recommend_outfits",
           "render_tryon"],
  "reply": "Here are 3 looks...",
  "cards": ["out_77c","out_78a",
            "out_79f"] }
```

Techs that may be used

Layer	Options
Agent framework	LangGraph (recommended: graph + state + checkpoints), CrewAI, OpenAI Agents SDK
LLM	Claude / GPT-4o-class via API for tool calling; open option: Llama 3.x / Qwen via Ollama or vLLM
Memory	Short-term: LangGraph state; long-term: Postgres + vector store
Guardrails	Pydantic tool schemas, refusal for off-topic (open-domain chatbot rejected in BO8)
Voice	Whisper (speech-to-text) for voice messages

3. Vision agent (member 1)

Role. Digitises garments: background removal, attribute classification, embedding. Also handles 'Outfit From a Photo' and Smart Buy photos.

Example input

```
{ "image": "s3://dressme/raw/
     itm_8f21.jpg",
  "mode": "add_to_wardrobe" }

mode ∈ {add_to_wardrobe,
        smartbuy_probe,
        inspiration_photo}
```

Example output

```
{ "category": "top",
  "subcategory": "shirt",
  "colors": [{"name":"navy",
              "share":0.82}],
  "pattern": "solid",
  "season": ["spring","summer"],
  "confidence": 0.91,
  "embedding": [0.012, -0.33,
                ... 512 floats] }
```

Techs that may be used

Step	Techs
Segmentation	rembg (U²-Net), SAM 2, or a YOLOv8-seg fine-tuned on clothes
Classification	Fine-tuned EfficientNet / ViT; zero-shot with FashionCLIP as baseline
Colour	K-means on cutout pixels (LAB space) + colour-name mapping
Embedding	FashionCLIP / Marqo-FashionSigLIP (512-d)
Datasets	DeepFashion2, Fashionpedia, iMaterialist + own Tunisian set (jebba, kachabia, modest wear)

4. Stylist agent (member 2)

Role. Builds outfits from the real wardrobe for a context (occasion, weather, style). Powers outfit of the day, Remix, 'complete with AI' in the builder.

Example input

```
{ "user_id": "usr_042",
  "occasion": "interview",
  "weather": {"temp_c":24},
  "style_vector": "vec_u042",
  "locked_items": ["itm_8f21"],
  "k": 3 }
```

Example output

```
{ "outfits": [
   {"items":["itm_8f21","itm_1a90",
             "itm_44de","itm_0b13"],
    "compatibility":0.86,
    "why":["formal","heat-friendly",
           "navy+beige harmony"]},
   ...] }
```

Techs that may be used

Step	Techs
Candidate filter	Rules: season, weather, formality range, category slots (top/bottom/shoes/outer)
Compatibility	Siamese / transformer outfit model trained on Polyvore Outfits; baseline: cosine on CLIP embeddings
Diversity	MMR re-ranking so the 3 proposals differ
Explanation	LLM fed with the scores and attributes (no free invention)
Context	Open-Meteo weather API, occasion taxonomy incl. Tunisian occasions (wedding, Aïd, Ramadan evening)

5. Profile & analytics agent (member 3)

Role. Learns the user's style from events, computes wardrobe analytics (wear rate, duplicates, gaps), feeds explanations.

Example input

```
{ "user_id": "usr_042",
  "events": [
   {"type":"outfit_feedback",
    "target_id":"out_77c",
    "value":"accept"},
   {"type":"worn",
    "target_id":"itm_1a90"}] }
```

Example output

```
{ "style_vector": [...512],
  "traits": {"formality":0.62,
             "palette":["navy","beige"],
             "avoid":["neon"]},
  "analytics": {
    "wear_rate": 0.41,
    "never_worn": ["itm_09aa"],
    "duplicates": [["itm_3","itm_7"]],
    "gaps": ["light jacket"] } }
```

Techs that may be used

Step	Techs
Style vector	Exponential moving average of accepted outfit embeddings minus rejected
Learning	Contextual bandit (LinUCB / Thompson) on top of Stylist scores; RL later
Cold start	Swipe onboarding on 15 generated looks
Analytics	SQL + pandas; duplicates = embedding similarity > 0.85
Scheduling	Celery beat nightly jobs

6. Smart Buy agent (member 4)

Role. Decides if a candidate piece complements or duplicates the wardrobe, and how many new outfits it unlocks.

Example input

```
{ "user_id": "usr_042",
  "probe_image": "s3://dressme/
     probe/p_311.jpg",
  "price_tnd": 89 }
```

Example output

```
{ "score": 78,
  "verdict": "good_buy",
  "duplicates": [],
  "new_outfits": 6,
  "pairs_with": ["itm_1a90",
                 "itm_44de"],
  "reasons": ["fills 'light jacket'
              gap", "matches palette"] }
```

Techs that may be used

Step	Techs
Similarity	Qdrant / pgvector k-NN on FashionCLIP embeddings
New outfits	Calls Stylist agent with the probe item locked
Score	Weighted: gap fill, compatibility, redundancy penalty, cost per wear (later)
Sharing	Shareable verdict card (BO7 poll)

7. Local shop agent (member 4)

Role. Answers 'where in Tunis can I find…?' from a curated base of Tunisian boutiques and brands (no live scraping).

Example input

```
{ "query": "white linen shirt
     under 80 TND in Tunis",
  "gap": "white shirt",
  "style_vector": "vec_u042" }
```

Example output

```
{ "results": [
   {"shop":"<partner boutique>",
    "area":"La Marsa",
    "item":"Linen shirt, white",
    "price_tnd":75,
    "match":0.81}] }
```

Techs that may be used

Step	Techs
Knowledge base	Curated CSV/JSON of shops and pieces, entered manually or from partner files
Retrieval	RAG: multilingual embeddings (e5 / bge-m3, handles FR/AR/EN) + hybrid BM25
Filters	Price (TND), city, category, style match
Framework	LlamaIndex or LangChain retriever as a LangGraph tool

8. Try-on agent (member 5)

Role. Shows outfits on a faceless 3D avatar built from the user's measurements. 2D diffusion try-on is the fallback.

Example input

```
{ "user_id": "usr_042",
  "body": {"height_cm":178,
           "chest_cm":98,
           "waist_cm":84,
           "hips_cm":96},
  "items": ["itm_8f21","itm_1a90"] }
```

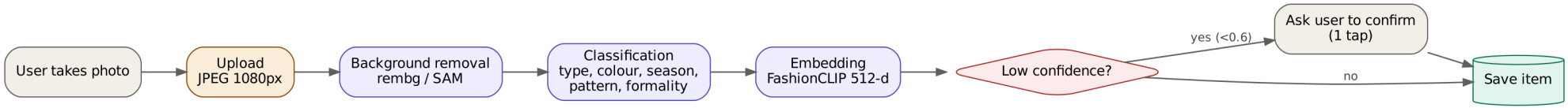
Example output

```
{ "avatar": "s3://dressme/
     avatar/usr_042.glb",
  "render": "s3://dressme/vto/
     out_77c.glb",
  "preview_png": ".../out_77c.png",
  "latency_ms": 4200 }
```

Techs that may be used

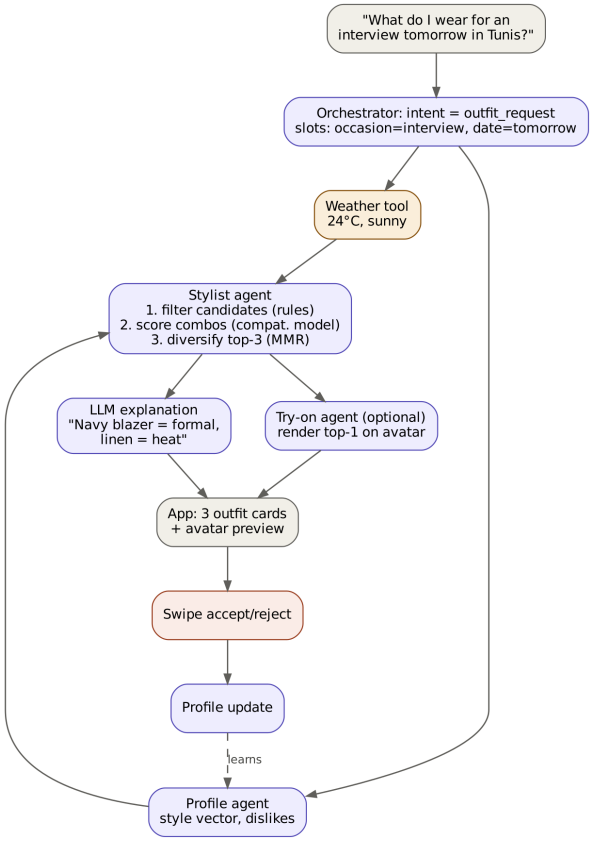
Step	Techs
Body model	SMPL-X shape fit from measurements
Garments	Template meshes per category (shirt, trousers, dress…) + texture from the cutout image
Rendering	glTF/GLB served to Flutter (model_viewer_plus) or Unity/Three.js view
Fallback	2D diffusion VTON: IDM-VTON / OOTDiffusion on GPU
Compute	GPU worker (Colab / cloud GPU) behind Celery queue

9. Flow — adding a garment



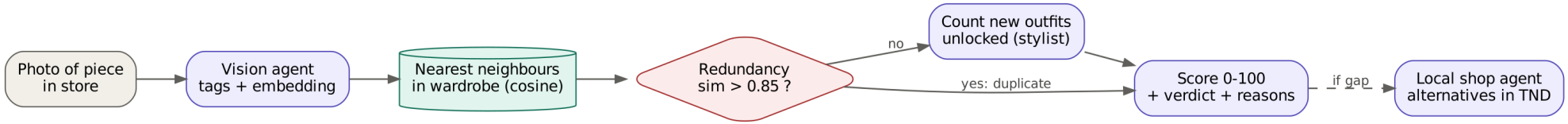
Synchronous part answers in < 3 s with a placeholder; segmentation and embedding finish in the queue. Low-confidence items get a one-tap confirmation, which also creates labelled training data.

10. Flow — outfit request (chat or outfit of the day)



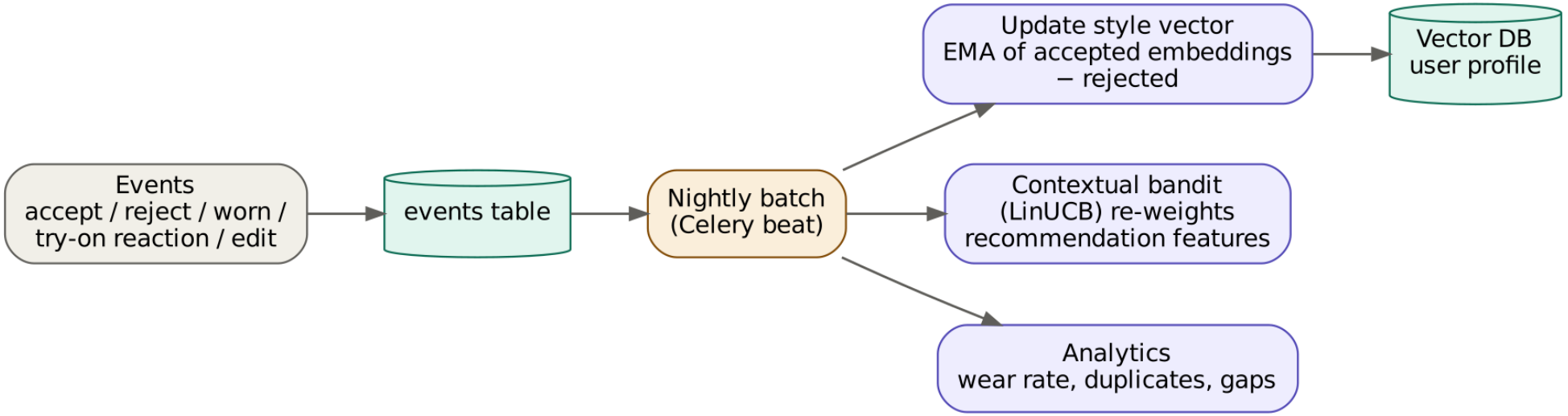
The same path serves the morning outfit (triggered by a scheduled job instead of a message) and chat requests. Every swipe becomes an event for the Profile agent.

11. Flow — Smart Buy check in store



Thresholds (0.85 similarity, 0.6 confidence) are starting values to tune on your validation set.

12. Flow — learning loop



Start with the EMA style vector (simple, explainable); add the bandit once you have enough events.

13. Global tech stack

Layer	Choice (recommended)	Alternatives
Mobile	Flutter	React Native
Backend API	FastAPI (Python)	Django REST
Agents	LangGraph	CrewAI, AutoGen
LLM	API model with tool calling	Llama / Qwen self-hosted (vLLM, Ollama)
Vision	PyTorch, rembg/SAM 2, FashionCLIP	YOLOv8-seg, timm models
Databases	PostgreSQL + pgvector, MinIO/S3	Qdrant, Firebase Storage
Async jobs	Celery + Redis	RQ, Dramatiq
MLOps	MLflow (experiments), DVC (datasets), Docker	Weights & Biases
Deployment	Docker Compose → cloud VM + GPU worker	Render, Railway, GCP Cloud Run
Monitoring	LangSmith / Langfuse (agent traces), Prometheus	Sentry

14. Ownership and data sets per member

Member	Agent / module	Data they own	Main datasets
1	Vision agent	garments, images	DeepFashion2, Fashionpedia, own Tunisian photos
2	Stylist agent	outfits	Polyvore Outfits, own accepted outfits
3	Profile & analytics	events, style vectors	App events (synthetic first)
4	Smart Buy + Local shop	shop knowledge base	Curated Tunisian shop list
5	Try-on agent	avatars, renders	SMPL-X, VITON-HD (fallback)
6	Orchestrator, API, MLOps	schemas, conversations	Test dialogues per BO